RAMNARAIN RUIA AUTONOMOUS COLLEGE
DEPARTMENT OF COMPUTER SCIENCE & IT (2025-26)

SEAT NO : 4221 **STUDENT NAME : Sandhya Shivdhar Gupta**

CLASS: F.Y.BSc. **SEMESTER: II** **COURSE CODE: RUSSECCSPE111**

COURSE NAME : PRACTICAL OF – OOP using JAVA

FACULTY IN-CHARGE : MS. KIRAN PRAJAPATI

INDEX

SR. NO.	DATE	PARTICULAR	SIGNATURE
1	18/11/2025	Basic Java Programs-I	
2	19/11/2025	Basic Java Programs-II	
3	25/11/2025	Methods	
4	29/11/2025	Strings	
5	9/12/2025	Arrays	
6	20/12/2025	Classes and Objects	
7	6/1/2026	Access Modifiers	
8	16/12/2025	Packages	
9	3/1/2026	Inheritance	
10	24/1/2026	Polymorphism	
11	27/1/2026	Exception Handling	
12	3/2/2026	File handling and Stream concepts	
13	7/2/2026	Socket Programming	
14	10/2/2026	Multithreading	

15	14/2/2026	Interfaces and Abstract classes	

Practical 1

1)Hello world code:

```
public class Prog1{

public static void main(String[] args){

System.out.println("Hello world");

}

}
```

Output :

```
C:\Users\Admin\Documents\Java>cd C:\Users\Admin\Documents\Java

C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
Hello world
```

2)find product of two floating numbers:

Code :

```
public class Prog1{

public static void main(String[] args){

float x=1.4f;

float y=1.4f;

System.out.println(x*y);

}

}
```

Output :

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
1.9599999
```

3)find a ASCII value of a number

Code :

```
public class Prog1{
public static void main(String[] args){
int num1=8;
char conversion=(char) (num1+'0');
int ASCIIdigit=conversion;
System.out.println(ASCIIdigit);
}
}
```

Output :

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
56

C:\Users\Admin\Documents\Java>
```

4)Swapping of two numbers:

Code :

```
public class Prog1{
public static void main(String[] args){
int num1=7;
int num2=11;
System.out.println("before swapping: "+num1+" "+num2);
int t=num1;
num1=num2;
num2=t;
System.out.println("after swapping: "+num1+" "+num2);
}
}
```

Output :

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
before swapping: 7 11
after swapping: 11 7
```

5)Compute Quotient and remainder

Code :

```
public class Prog1{

public static void main(String[] args){

int num1=15;

int num2=5;

System.out.println("Quotient is: "+15/5);

System.out.println("Remainder "+15%5);

}

}
```

Output :

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
Quotient is: 3
Remainder 0
```

6)check whether a number is even or odd

Code :

```
public class Prog1{

public static void main(String[] args){

int num1=15;

if(num1%2==0){

System.out.println("EVEN");

}

else

{
```

```
System.out.println("ODD");
}
}
}
```

Output:

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
ODD
```

7)Find greatest of 3 numbers:

Code :

```
public class Prog1{
public static void main(String[] args){
int num1=15;
int num2=4;
int num3=100;
int max=num1;
if(max<=num2){
max=num2;
}
if(max<=num3){
max=num3;
}
System.out.println("maximum is "+max);
}
}
```

Output:

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
maximum is 100
```

8)whether positive negative or zero

Code :

```
public class Prog1{

public static void main(String[] args){

int num1=15;

if(num1>0){

System.out.println("Positive");

}

else if(num1<0){

System.out.println("Negative");

}

else

{

System.out.println("Zero");

}

}

}
```

Output :

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
Positive
```

9)Vowel or consonant :

code :

```
public class Prog1{

public static void main(String[] args){
```

```
char alpha='b';

if(alpha=='a'||alpha=='e' ||alpha== 'i' ||alpha== 'o' ||alpha== 'u' ||alpha== 'A' ||alpha==
'E' ||alpha== 'I' ||alpha== 'O' ||alpha== 'U')

{

System.out.println("Vowel");

}

else

{

System.out.println("Consonant");

}

}

}
```

Output :

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
Consonant
```

10)check whether a person is eligible to vote or not

Code :

```
public class Prog1{

public static void main(String[] args){

int age=17;

if(age>=18)

{

System.out.println("Eligible");

}

else

{

System.out.println("not eligible");
```

```
}

}

}
```

Output :

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
not eligible
```

BASIC PROGRAM USING CONDITIONAL AND LOOPING STATEMENT

1.SUM OF FIRST 10 NATURAL NUMBERS

Code :

```
public class Prog1{
public static void main(String[] args){
int sum=0,i=1;
while(i<=10){
sum=sum+i;
i++;
}
System.out.println(sum);
}
}
```

Output:

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
55
```

2.MULTIPLICATION TABLE OF 5

Code :

```
public class Prog1{
public static void main(String[] args){
int i=1;
int prod=0;
while(i<=10){
prod=5*i;
System.out.println(prod);
```

```
i++;

}

}

}
```

Output:

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
5
10
15
20
25
30
35
40
45
50
```

3.FACTORIAL OF A NUMBERT

Code :

```
public class Prog1{

public static void main(String[] args){

int i=6;

int fact=1;

while(i!=0){

fact=fact*i;

i--;

}

System.out.println(fact);

}

}
```

Output:

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
720

C:\Users\Admin\Documents\Java>
```

4.FIBONACCI SERIES

Code :

```
public class Prog1{

public static void main(String[] args){

int a=0,b=1;

int c;

int n=10,i=1;

System.out.print(a+" "+b+" ");

while(i<=n-2){

c=a+b;

System.out.print(c+" ");

a=b;

b=c;

i++;

}

}

}
```

Output:

```
C:\Users\Admin\Documents\Java>java Prog1
0 1 1 2 3 5 8 13 21 34
C:\Users\Admin\Documents\Java>
```

5.PRINT ALPHABETS FROM A TO Z

Code :

```
public class Prog1{
public static void main(String[] args){
char ch;
for(ch='A';ch<='Z';ch++){
System.out.println(ch);
}
}
}
```

Output:

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
A
B
C
D
E
F
G
H
I
J
K
L
M
N
O
P
Q
R
S
T
U
V
W
X
Y
Z
```

6.DISPLAY THE NO. OF DIGITS IN A NUMBER

Code :

```
public class Prog1{
public static void main(String[] args){
int n=23;
int count=0;
int r;
while(n!=0){
r=n%10;
count++;
n=n/10;
}
System.out.println(count);
}
}
```

Output:

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
2

C:\Users\Admin\Documents\Java>
```

7.DISPLAY THE SUM OF DIGITS IN A NUMBER

Code :

```
public class Prog1{
public static void main(String[] args){
int n=23;
int count=0;
int sum=0,r;
while(n!=0){
r=n%10;
```

```
sum=sum+r;

n=n/10;

}

System.out.println(sum);

}

}
```

Output:

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
5

C:\Users\Admin\Documents\Java>
```

8.REVERSE THE NUMBER

Code :

```
public class Prog1{

public static void main(String[] args){

int n=23;

int count=0;

int sum=0,r,rev=0;

int x=n;

while(n!=0){

r=n%10;

rev=(rev*10)+r;

n=n/10;

}

System.out.println(x);

System.out.println(rev);

}

}
```

Output :

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
23
32

C:\Users\Admin\Documents\Java>
```

9 .SIMPLE CALCULATOR USING SWITCH CASE

Code :

```
public class Prog1{

public static void main(String[] args){

int a=10,b=2;

System.out.println(a+b);

System.out.println(a-b);

System.out.println(a*b);

System.out.println(a/b);

System.out.println(a%b);

}

}
```

Output:

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
12
8
20
5
0

C:\Users\Admin\Documents\Java>
```

10.PRIME NO. OR NOT

Code :

```
public class Prog1{
```

```
public static void main(String[] args){
int n=8;
int check=0;
for(int i=2;i<n;i++)
{
if(n%i==0){
check++;
}
}
if(check==0){
System.out.println("Prime number");
}
else{
System.out.println("Not Prime number");
}
}
}
```

Output:

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
Not Prime number

C:\Users\Admin\Documents\Java>
```

11.prime number in a RANGE

Code :

```
public class Prog1{
public static void main(String[] args){
System.out.println("List of Prime number");
```

```
int n=50;
for(int i=2;i<n;i++)
{
  int check=0;
  for(int j=2;j<i;j++)
   {
     if(i%j==0){
     check++;
     break;
     }
   }
 if(check==0){
  System.out.println(i);
  }
}
}
}
```

Output :

```
C:\Users\Admin\Documents\Java>java Prog1
List of Prime number
2
3
5
7
11
13
17
19
23
29
31
37
41
43
47

C:\Users\Admin\Documents\Java>
```

12.armstrong or not

Code :

```
public class Prog1{

public static void main(String[] args){

int n=153;

int sum=0;

int x=n,r;

while(n!=0){

r=n%10;

sum=sum+(r*r*r);

n=n/10;

}

if(sum==x){

System.out.println("armstrong number");

}

else{
```

```
System.out.println("Not armstrong number");

}

}

}
```

Output:

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
armstrong number

C:\Users\Admin\Documents\Java>
```

13.PALINDROME OR NOT

Code :

```
public class Prog1{

public static void main(String[] args){

int n=121;

int count=0;

int sum=0,r,rev=0;

int x=n;

while(n!=0){

r=n%10;

rev=(rev*10)+r;

n=n/10;

}

System.out.println("original no.:"+x);

System.out.println("reversed no:"+rev);

if(rev==x){

System.out.println("palindrome");

}

else{
```

```
System.out.println("not palindrome");

}

}

}
```

Output:

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
original no.:121
reversed no:121
palindrome

C:\Users\Admin\Documents\Java>
```

PRACTICAL 3:JAVA METHODS

Q1. Write a program to display four mathematical operations using methods

Code :

```
public class Mymethod{
  static void add(int x,int y){
   System.out.println("addition is:"+(x+y));
  }
  static void sub(int x,int y){
  System.out.println("Subtraction is:"+(x-y));
  }
  static void mult(int x,int y){
  System.out.println("multiplication is:"+(x*y));
  }
  static void div(int x,int y){
  System.out.println("Division is:"+(x/y));
  }
  public static void main(String[] args){
   add(5,8);
   sub(15,8);
   mult(6,9);
   div(16,2);
  }
}
```

Output:

```
C:\Users\Admin\Documents\Java>javac Mymethod.java

C:\Users\Admin\Documents\Java>java Mymethod
addition is:13
Subtraction is:7
multiplication is:54
Division is:8
```

Q2.write a program to check whether the age is graeter than 18 or not using methods.

Code :

```
public class Mymethod{
  static void check_age(int age){
   if(age>=18){
   System.out.println("eligible");
  }
  else{
   System.out.println("Not eligible");
 }
}
   public static void main(String[] args){
   check_age(19);
  }
}
```

Output:

```
C:\Users\Admin\Documents\Java>javac Mymethod.java

C:\Users\Admin\Documents\Java>java Mymethod
eligible
```

Q3. Write a program to display the volume of cylinder, sphere, cuboid using method overloading

code :

```
public class Mymethod{
  static void volume(float r,float h){
   System.out.println("Volume of the cylinder is :"+(3.142*r*r*h));
 }
  static void volume(float r){
   System.out.println("Volume of the sphere is :"+(4/3*3.142*r*r*r));
 }
  static void volume(float l,float b,float h){
   System.out.println("Volume of the cuboid is :"+(l*b*h));
 }
 public static void main(String[] args){
 volume(2,4);
 volume(2);
 volume(2,4,8);
}
}
```

Output:

```
C:\Users\Admin\Documents\Java>javac Mymethod.java

C:\Users\Admin\Documents\Java>java Mymethod
Volume of the cylinder is :50.272
Volume of the sphere is :25.136
Volume of the cuboid is :64.0
```

Q4.write a program to find the smallest among 3 numbers. Take input from users.

CODE :

```
import java.util.Scanner;
```

```
public class Smallestnum{
public static int smallest(int a,int b,int c){
int min=a;
if(min>=b){
min=b;
}
if(min>=c){
min=c;
}
return min;
}
public static void main(String[] args)
{
Scanner obj= new Scanner(System.in);
System.out.println("enter first number:");
int num1=obj.nextInt();
System.out.println("enter second number:");
int num2=obj.nextInt();
System.out.println("enter third number:");
int num3=obj.nextInt();
int result=smallest(num1,num2,num3);
System.out.println(result);
}
}
```

OUTPUT:

```
C:\Users\Admin\Documents\Java>javac Smallestnum.java

C:\Users\Admin\Documents\Java>java Smallestnum
enter first number:
15
enter second number:
11
enter third number:
3
3
```

Q5.write a program to calculate the average of 3numbers.

CODE :

```
public class Mymethod{

  static void avg(float a,float b,float c){

   System.out.println("Average is :"+((a+b+c)/3));

 }

 public static void main(String[] args){

  avg(15,7,34);

}

}
```

OUTPUT:

```
C:\Users\Admin\Documents\Java>javac Mymethod.java

C:\Users\Admin\Documents\Java>java Mymethod
Average is :18.666666
```

Q6. Write a program count all vowels in a string.

CODE:

```
import java.util.Scanner;

public class Vowels{

public static void main(String[] args){

Scanner obj = new Scanner(System.in);
```

```
System.out.println("Enter a word:");

String word = obj.next();

char[] letters = word.toCharArray();

int i = 0;

for(char ch : letters){

    if(ch=='a'||ch=='e'||ch=='i'||ch=='o'||ch=='u'||ch=='A'||ch=='E'||ch=='I'||ch=='O'||
ch=='U'){

    i = i + 1;

   }

  }

System.out.println("The number of vowels are :");

System.out.println(i);

}

}
```

OUTPUT:

```
C:\Users\admin\OneDrive\Documents>javac Vowels.java

C:\Users\admin\OneDrive\Documents>java Vowels
Enter a word:
sandhya
The number of vowels are :
2

C:\Users\admin\OneDrive\Documents>
```

Q7.write a program to display whether the year is a leap year or not.

CODE:

```
public class Mymethod{

  static void leap_year(int year){

   if(year%4==0 && year%100!=0 || year%400==0){
```

```
  System.out.println("leap year");
}
else{
   System.out.println("Not leap year");
}
 }
 public static void main(String[] args){
  leap_year(2020);
}
}
```

OUTPUT:

```
C:\Users\Admin\Documents\Java>javac Mymethod.java

C:\Users\Admin\Documents\Java>java Mymethod
leap year
```

Q8.write a program to display the circumference and area of circle.

CODE :

```
public class Mymethod{
  static void area_circum(float r){
  System.out.println("Area of circumference"+(2*3.142*r));
}
  static void area_circle(float r){
  System.out.println("Area of circle"+(3.142*r*r));
}
 public static void main(String[] args){
  area_circum(4.5f);
  area_circle(2.5f);
}
```

}

OUTPUT:

```
C:\Users\Admin\Documents\Java>javac Mymethod.java

C:\Users\Admin\Documents\Java>java Mymethod
Area of circumference28.278
Area of circle19.6375
```

Q9.write a program to calculate the area of triangles(right angled triangle, equilateral triangle, isosceles triangle , scalene triangle).

CODE :

```
public class Mymethod{
  static void right_angled_triangle(float b,float h){
  System.out.println("Area of right_angled_triangle : "+(0.5*b*h));
}
  static void equilateral_triangle(float s){
  System.out.println("Area of equilateral_triangle : "+((Math.sqrt(3)/4)*s*s));
}
  static void isosceles_triangle(float b,float h){
  System.out.println("Area of isosceles_triangle: "+(0.5*b*h));
}
  static void scalene_triangle(float b,float h){
  System.out.println("Area of scalene_triangle: "+(0.5*b*h));
}
 public static void main(String[] args){
 right_angled_triangle(2.5f,6.5f);
 equilateral_triangle(4f);
 isosceles_triangle(4.8f,1.2f);
 scalene_triangle(2.2f,3.2f);
}
```

}

OUTPUT:

```
C:\Users\Admin\Documents\Java>javac Mymethod.java

C:\Users\Admin\Documents\Java>java Mymethod
Area of right_angled_triangle : 8.125
Area of equilateral_triangle : 6.928203230275509
Area of isosceles_triangle: 2.8800002288818405
Area of scalene_triangle: 3.520000128746034
```

PRACTICAL 4

TAKE INPUTS FROM USER (STRING INPUT)

CODE :

```
import java.util.Scanner;

public class Mystring{

public static void main(String[] args){

Scanner obj=new Scanner(System.in);

System.out.println("enter first string:");

String txt=obj.nextLine();

System.out.println("enter second string:");

String txt1=obj.nextLine();

}

}
```

CODE :

```
CODE import java.util.Scanner;

public class Mystring{

public static void main(String[] args){

Scanner obj=new Scanner(System.in);

System.out.print("enter first string:");

String txt=obj.nextLine();

System.out.print("enter second string:");

String txt1=obj.nextLine();


System.out.println("1.the length of the string is:"+txt.length()+"\n");
```

```
System.out.println("2.the lowercase of the first string is:"+txt.toLowerCase()
+"\n");

System.out.println("3.the uppercase of the second string is:"+txt.toUpperCase()
+"\n");

System.out.println("4.the index of the string 'hello' is:"+txt1.indexOf("hello")
+"\n");

System.out.println("5.The character at the specific index:"+txt.charAt(6));

System.out.println("6.Concatenate ' got it' with the second
string:"+txt1.concat(" got it"));

System.out.println("7.Check whether the 'sandhya' is present in first string or
not if yes then it will return true else false:"+txt.contains("sandhya"));

System.out.println("8.check whether the second string has 0 length or not if it
has it will return true else false :"+txt.isEmpty());

System.out.println("9.tell the index of last occurrence of 's' in second
string:"+txt.lastIndexOf('s'));

System.out.println("10.tell the index of last occurrence of 's' in second
string:"+txt.lastIndexOf('s'));

System.out.println("11.tell if the first string ends with 'yes' or not it will return
true / false:"+txt.endsWith("yes"));
```

```
System.out.println("12.Replace 's' with 'x' in second
string:"+txt1.replace('s','x'));


System.out.println("13.tell if the second string ends with 'No' or not it will
return true / false:"+txt.startsWith("No"));


System.out.println("14.Subsequence from index 5 to index
10:"+txt.subSequence(5,10));


System.out.println("15.trim() removes all whitespaces:"+txt1.trim());
}
}
```

OUTPUT:

```
C:\Users\Admin\Documents\Java>java Mystring
enter first string:sandhya gupta
enter second string:i will not say hello to everyone
1.the length of the string is:13

2.the lowercase of the first string is:sandhya gupta

3.the uppercase of the second string is:SANDHYA GUPTA

4.the index of the string 'hello' is:15

5.The character at the specific index:a
6.Concatenate ' got it' with the second string:i will not say hello to everyone got it
7.Check whether the 'sandhya' is present in first string or not if yes then it will return true else false:true
8.check whether the second string has 0 length or not if it has it will return true else false :false
9.tell the index of last occurrence of 's' in second string:0
10.tell the index of last occurrence of 's' in second string:0
11.tell if the first string ends with 'yes' or not it will return true / false:false
12.Replace 's' with 'x' in second string:i will not xay hello to everyone
13.tell if the second string ends with 'No' or not it will return true / false:false
14.Subsequence from index 5 to index 10:ya gu
15.trim() removes all whitespaces:i will not say hello to everyone
```

PRACTICAL 5

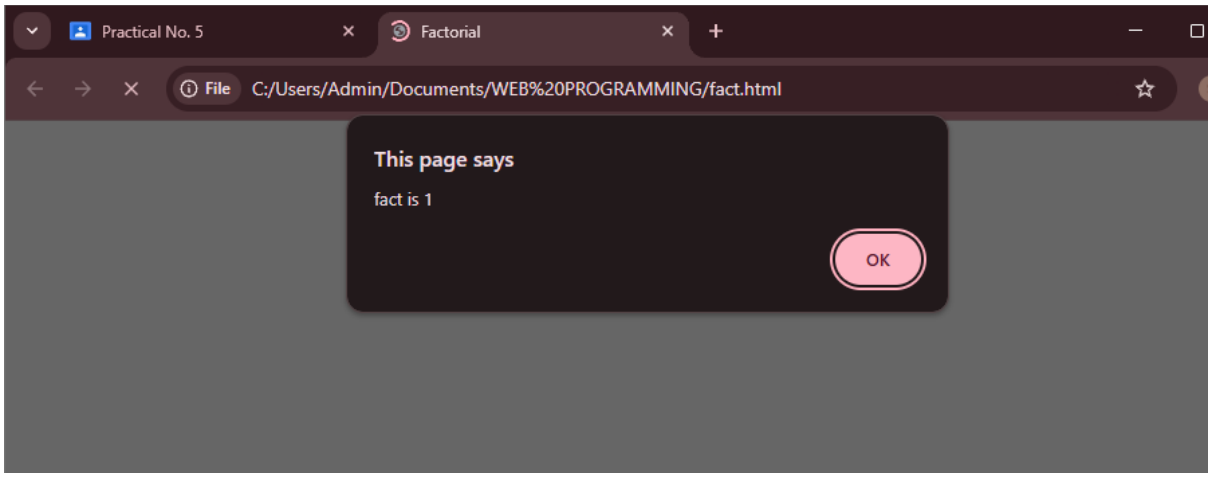
1)factorial

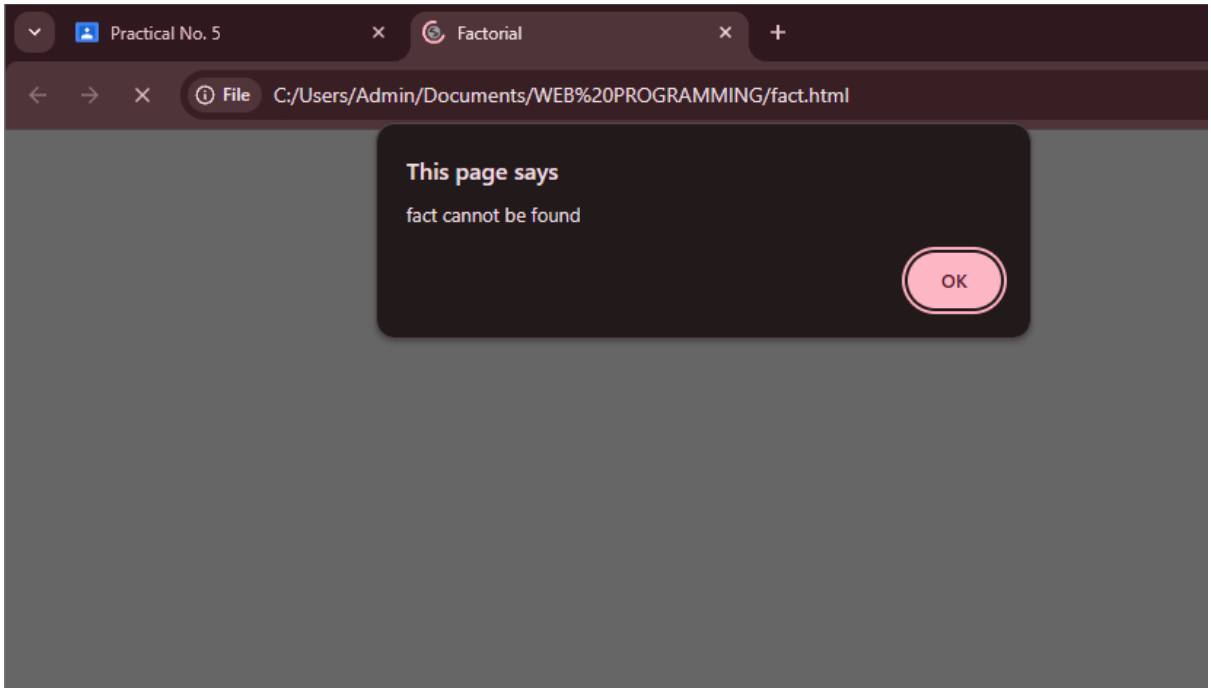
Code :

```
<html>
<head>
<title>Factorial</title>
</head>
<script language ="javascript">
var fact=1,i,n;
i=window.prompt("Find factorial of entered no:");
n=window.eval(i);
document.write("entered num="+n);
if(n<0)
{
alert("fact cannot be found");
}
else
{
if(n==0)
{
alert("fact is 1");
}
else
{
```

```
 for(i=1;i<=n;i++)
{
 fact=fact*i;
}
document.write("<br>fact="+fact);
}
}
</script>
</body>
</html>
```

Output :





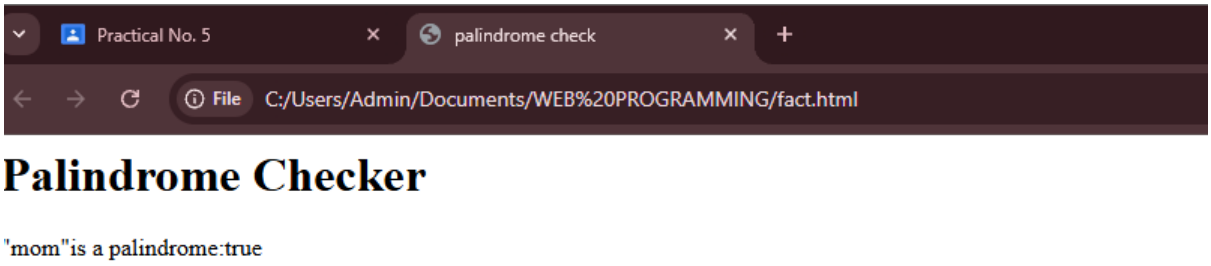
2)palindrome

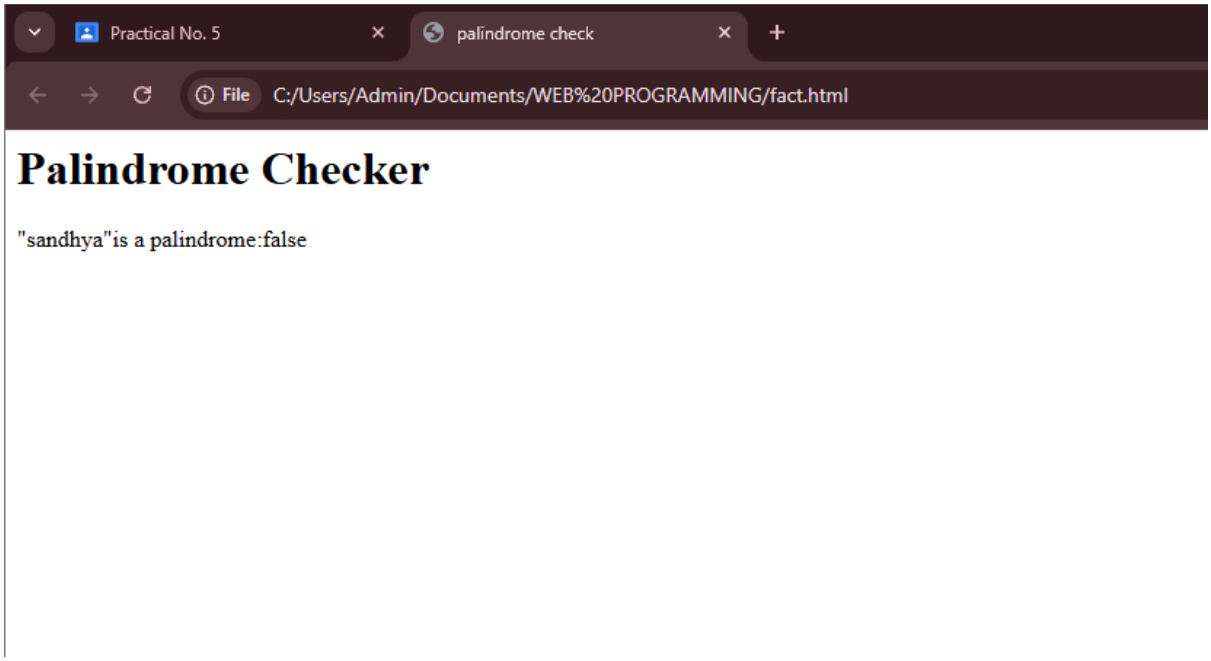
Code :

```
<html>
<head>
<title>
palindrome check
</title>
</head>
<body>
<h1>Palindrome Checker</h1>
<p id="result"></p>
<script language="javascript">
function isPalindrome(str)
{
str=str.toString().replace(/[^a-zA-Z0-9]/g,"").toLowerCase();
```

```
let reversedStr=str.split("").reverse().join("");

return str===reversedStr;

}

let userinput = prompt("Enter a word or phrase to check whether it's a palindrome:");


document.getElementById("result").innerText=`"${userinput}"is a palindrome:`+isPalindrome(userinput);

</script>

</body>

</html>
```

Output :





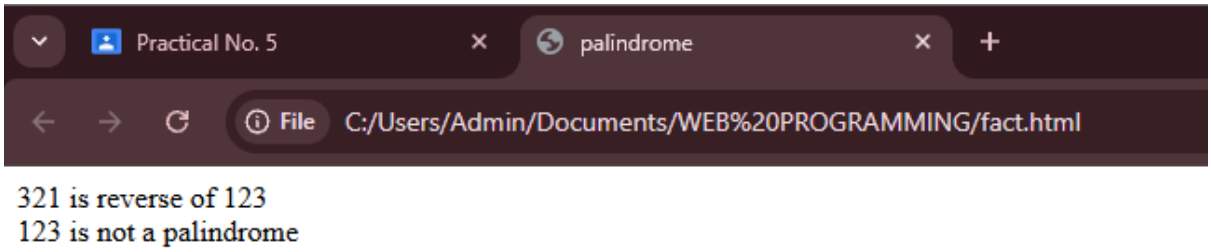
3) palindrome of number string

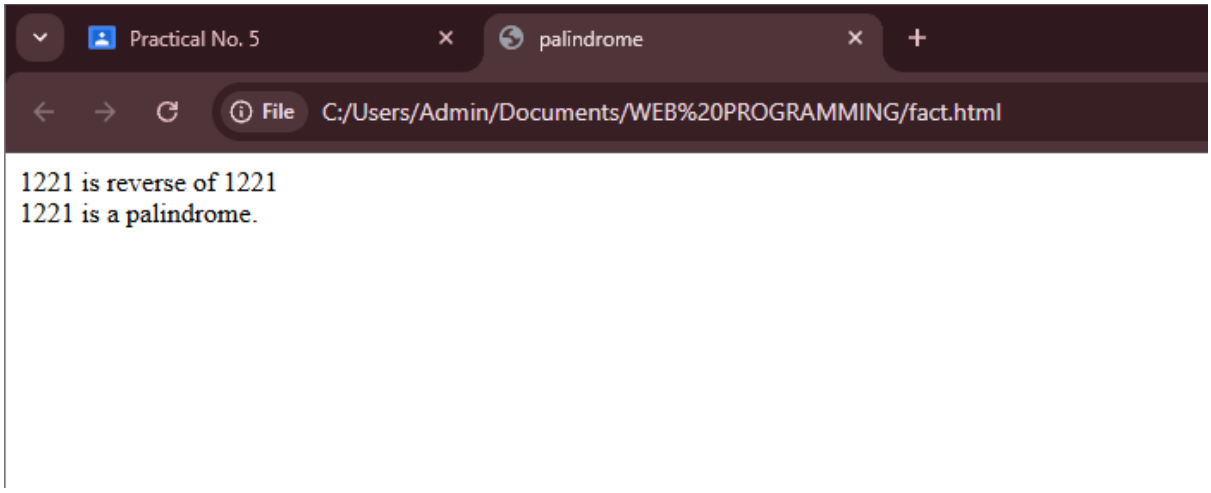
Code :

```
<html>
<head>
<title>
palindrome
</title>
</head>
<body>
<script language="javascript">
var i,n,j,k,r;
i=window.prompt("Check number is palindrome or not","enter a number");
n=window.eval(i);
k=n;
r=0;
while(n>0)
```

```
{
j=parseInt(n%10);
n=parseInt(n/10);
r=(r*10)+j;
}
document.write(r+" is reverse of "+k+"<br>");
if(r==k)
document.write(k+" is a palindrome."+"<br>");
else
document.write(k+" is not a palindrome"+"<br>");
</script>
</body>
</html>
```

Output:





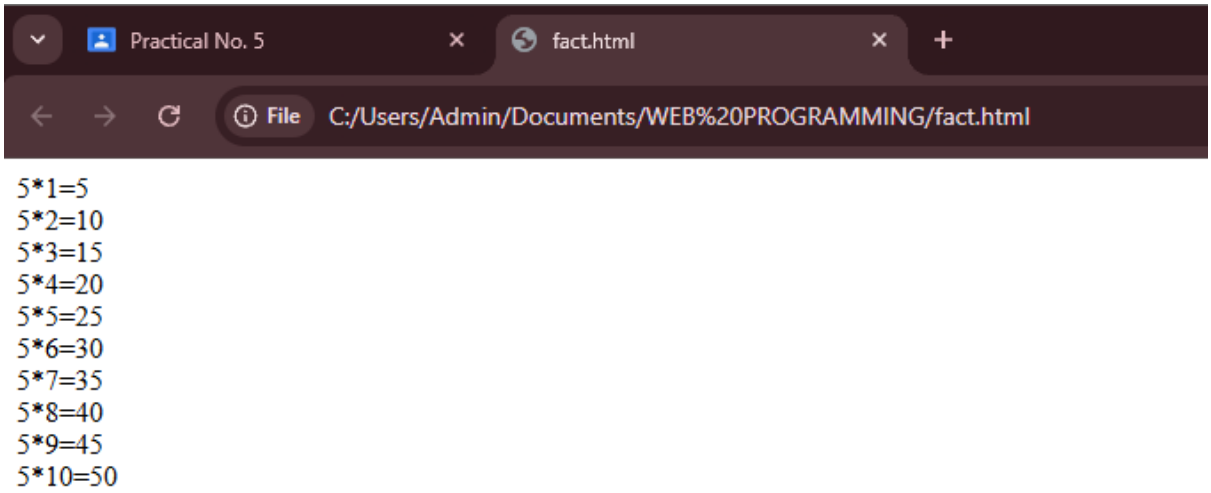
4)Multiplication table

Code :

```
<html>
<head>
<title>
Multiplication table
</title>
</head>
<body>
<script type="text/javascript">
function multi()
{
var num=prompt("please enter number","multiplication table");
for(i=1;i<=10;i++)
document.writeln(num+"*"+i+"="+num*i+"<br>");
}
</script>
<body onLoad="multi()">
```

```
</body>

</html>
```

Output:



PRACTICAL 6

1)Implementing any 5 built-in packages.

CODE :
Code:

```
import java.time.LocalDate;
public class Prog1{
public static void main(String [] args){
LocalDate today=LocalDate.now();
System.out.println("Today's Date: "+today);
}
}
```

OUTPUT :

```
C:\Users\admin>cd C:\Users\admin\OneDrive\Documents

C:\Users\admin\OneDrive\Documents>javac P1.java
P1.java:3: error: <identifier> expected
public static void main(String[] *args){
                                 ^
1 error

C:\Users\admin\OneDrive\Documents>javac P1.java

C:\Users\admin\OneDrive\Documents>java P1
Today's Date: 2025-12-19
```

2)
CODE :
Code:

```
public class Prog1{
public static void main(String [] args){
System.out.println("The absolute value of -5 is : "+Math.abs(-5));
System.out.println("The minimum of 10 and 45 is : "+Math.min(10,45));
System.out.println("The power of 2 raised to 3 is : "+Math.pow(2,3));
System.out.println("The square root of 1225 is : "+Math.sqrt(1225));
System.out.println("The value of log 35 to base 10 is : "+Math.log10(35));
```

```
}
}
```

OUTPUT :

```
C:\Users\admin\OneDrive\Documents>javac P1.java

C:\Users\admin\OneDrive\Documents>java P1
The absolute value of -5 is : 5
The minimum of 10 and 45 is : 10
The power of 2 raised to 3 is : 8.0
The square root of 1225 is : 35.0
The value of log 35 to base 10 is : 1.5440680443502757
```

3)

CODE :

PART 1:

```
package Result;
import java.util.*;
public class P1{
public void student(String nickname,int marks){
System.out.println("Sandhya nickname is "+nickname);
System.out.println("she got "+marks+" marks");
}
}
```

PART 2:

```
import Result.P1;
public class P2{
public static void main(String[] args){
P1 s1=new P1();
s1.student("sandy",100);
}
}
```

OUTPUT :

```
C:\Users\admin\OneDrive\Documents>javac P1.java

C:\Users\admin\OneDrive\Documents>javac -d . P1.java

C:\Users\admin\OneDrive\Documents>javac P2.java

C:\Users\admin\OneDrive\Documents>java P2
Sandhya nickname is sandy
she got 100 marks
```

4)
CODE :
PART 1:

(Two classes in Result packages)

```
package Result;
import java.util.*;
public class V1{
public void volunteer(int count,String name){
System.out.println(name+" is a volunteer "+"and there are "+count+"
volunteers");
}
}

package Result;
import java.util.*;
public class T1{
public void teacher(String subject,int age){
System.out.println("Siya maam teaches "+subject+" and her age is
"+age);
}
}
```

PART 2:(Importing both the classes in one file)

```
import Result.P1;
import Result.V1;
import Result.T1;
public class P2{
public static void main(String[] args){
```

```
P1 s1=new P1();
s1.student("sandy",100);

V1 v1=new V1();
v1.volunteer(38,"Priyanka");

T1 t1=new T1();
t1.teacher("Web",42);
}
}
```

OUTPUT:

```
C:\Users\admin\OneDrive\Documents>javac V1.java

C:\Users\admin\OneDrive\Documents>java -d . V1.java
. not found

C:\Users\admin\OneDrive\Documents>javac -d . V1.java
```

```
C:\Users\admin\OneDrive\Documents>javac T1.java

C:\Users\admin\OneDrive\Documents>javac -d . T1.java
```

```
C:\Users\admin\OneDrive\Documents>javac P2.java

C:\Users\admin\OneDrive\Documents>java P2
Sandhya nickname is sandy
she got 100 marks
Priyanka is a volunteer and there are 38 volunteers
Siya maam teaches Web and her age is 42

C:\Users\admin\OneDrive\Documents>
```

5)

code :

```
import java.time.LocalTime;
public class Time{
public static void main(String[] args){
LocalTime t=LocalTime.now();
System.out.println("Time is:"+t);
}
}
```

output :

```
C:\Users\admin\OneDrive\Documents>javac Time.java

C:\Users\admin\OneDrive\Documents>java Time
Time is:19:35:11.023739500

C:\Users\admin\OneDrive\Documents>java Time
Time is:19:35:16.212480900
```

6)

code :

```
import java.util.Arrays;
public class Time{
public static void main(String[] args){
int[] arr={7,4,9,1,3};
Arrays.sort(arr);
System.out.println("Sorted array is :"+Arrays.toString(arr));
}
}
```

Output:

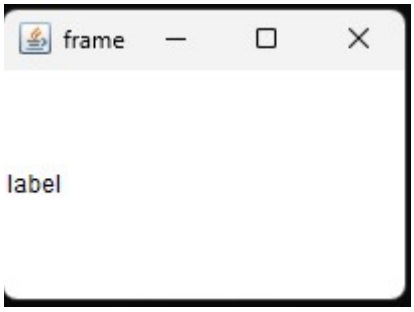
```
C:\Users\admin\OneDrive\Documents>javac Time.java

C:\Users\admin\OneDrive\Documents>java Time
Sorted array is :[1, 3, 4, 7, 9]

C:\Users\admin\OneDrive\Documents>
```

7)Frames and labels

code:

```
import java.awt.Frame;
import java.awt.Label;
public class Time{
public static void main(String[] args){
Frame frame=new Frame("frame");
Label label=new Label("label");
frame.add(label);
frame.setSize(500,100);
frame.setVisible(true);
}
}
```

output:



PRACTICAL 7

Note :The final keyword is useful when you want a variable to always store the same value, like PI (3.14159...)

1)

Code :

```
class Student{
String name;
int roll_no;
}

class Ans{
public static void main(String[] args){
Student s = new Student();
s.name="John";
s.roll_no=4221;
System.out.println("Name is "+s.name+" and roll number is "+s.roll_no);
}
}
```

Output:

```
C:\Users\Admin>cd "C:\Users\Admin\Downloads"

C:\Users\Admin\Downloads>javac Ans.java

C:\Users\Admin\Downloads>java Ans
Name is John and roll number is 4221

C:\Users\Admin\Downloads>
```

2)

Code :

```
class Triangle{
int a,b,c;
public double area(){
  double s=(a+b+c)/2.0;
  return Math.pow((s*(s-a)*(s-b)*(s-c)),0.5);
}
public double perimeter(){
return (a+b+c)/2.0;
}
}
```

```
class Ans{
public static void main(String[] args){
Triangle t=new Triangle();
t.a=3;
t.b=4;
t.c=5;
System.out.println("Area of the triangle "+t.area());
System.out.println("Area of the triangle "+t.perimeter());
}
}
```

Output :

```
C:\Users\Admin\Downloads>javac Ans.java

C:\Users\Admin\Downloads>java Ans
Area of the triangle 6.0
Area of the triangle 6.0
```

3)

Code :

```
import java.util.*;
class Complex{
int real;
int imag;
public Complex(int r,int i){
real=r;
imag=i;
}
public static Complex add(Complex a,Complex b)
{
 return new Complex((a.real+b.real),
(a.imag+b.imag));
}
public static Complex diff(Complex a,Complex b){
 return new Complex((a.real-b.real),(a.imag-
b.imag));
}
```

```
public static Complex product(Complex
a,Complex b){
 return new Complex(((a.real*b.real)-
(a.imag*b.imag)),((a.real*b.imag)-
(a.imag*b.real)));
}

public void printComplex(){
 if(real==0 && imag!=0){
 System.out.println(imag+"i");
}
else if(imag==0 && real!=0){
 System.out.println(real);
}
else{
System.out.println(real+"+"+imag+"i");
}
```

```
}
}

class Ans{
public static void main(String[] args){
Complex c = new Complex(4,5);
Complex d = new Complex(9,4);
Complex e = Complex.add(c,d);
Complex f = Complex.diff(c,d);
Complex g = Complex.product(c,d);

e.printComplex();
f.printComplex();
g.printComplex();
}
}
```

Output :

```
C:\Users\Admin\Downloads>javac Ans.java

C:\Users\Admin\Downloads>java Ans
13+9i
-5+1i
16+-29i
```

4)

Code :

Output :

PRACTICAL 8

1. Write a program using Single inheritance. Create a vehicle class with fields for make and model, then create a car class that inherits from vehicle class and add a type field, Eg: SUV etc. Implement a method in each class to display information about the vehicle.

CODE :

```
class Vehicle{
String make;
String model;
Vehicle(String make,String model){
this.make=make;
this.model=model;
}
void display_info(){
System.out.println("Vehicle Make: "+make+" and Model: "+model);
}
}
class Car extends Vehicle{
String type;
Car(String make,String model,String type){
super(make,model);
this.type=type;
}
void displayCarInfo(){
display_info();
System.out.println("Vehicle Type: "+type);
}
}
public class Prog1{
```

```
public static void main(String[] args){

Car car1= new Car("Mach-e","GT","Sports Car");

car1.displayCarInfo();

}

}
```

OUTPUT :

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
Vehicle Make: Mach-e and Model: GT
Vehicle Type: Sports Car

C:\Users\Admin\Documents\Java>
```

2. Create a person class with name and age. Create a Student class that inherits from Person and adds a studentId. Then, create a Graduate class that inherits from Student and adds a degree field. Implement methods to display information about each class.

CODE :

```
class Person{

String name;

int age;

Person(String name,int age){

this.name=name;

this.age=age;

}

void display_person(){

System.out.println("The name is "+name+" and the age is "+age);

}

}

class Student extends Person{

int id;
```

```
Student(String name,int age,int id){
super(name,age);
this.id=id;
}
void display_student(){
display_person();
System.out.println("the id of the student is "+id);
}
}
class Graduate extends Student{
String degree;
Graduate(String name,int age,int id,String degree){
super(name,age,id);
this.degree=degree;
}
void display_graduate(){
display_student();
System.out.println("The degree of the student is "+degree);
}
}
public class Prog1{
public static void main(String[] args){
Graduate g1=new Graduate("Sandhya",18,4221,"BSC CS");
g1.display_graduate();
}
}
```

OUTPUT:

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
The name is Sandhya and the age is 18
the id of the student is 4221
The degree of the student is BSC CS

C:\Users\Admin\Documents\Java>
```

3. Create a shape class with a name field and add a method to display the shape name. Then, create two subclasses: Circle and Rectangle which inherit from Shape and each have method to calculate their respective area. Implement a program to demonstrate the functionality.

CODE :

```
class Shape{
String field;
Shape(String field){
this.field=field;
}
void display_shape_name(){
System.out.println("The name of the shape is "+field);
}
}
class Circle extends Shape{
int r;
Circle(String field,int r){
super(field);
this.r=r;
}
void get_circle_area(){
System.out.println("The area of the circle is "+(3.142*r*r));
}
}
```

```
class Rectangle extends Shape{
int l;
int b;
Rectangle(String field,int l,int b){
super(field);
this.l=l;
this.b=b;
}
void get_rectangle_area(){
System.out.println("The area of the rectangle is "+(l*b));
}
}
public class Prog1{
public static void main(String[] args){
Circle c1=new Circle("circle",3);
c1.display_shape_name();
c1.get_circle_area();
Rectangle r1=new Rectangle("rectangle",5,8);
r1.display_shape_name();
r1.get_rectangle_area();
}
}
```

OUTPUT:

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
The name of the shape is circle
The area of the circle is 28.278
The name of the shape is rectangle
The area of the rectangle is 40

C:\Users\Admin\Documents\Java>
```

4. Create a class Employee with a field salary. Create a subclass Programmer that adds a field bonus. Write a program to display the total income (salary + bonus) of a programmer.

CODE :

```
class Employee{
int salary;
Employee(int salary){
this.salary=salary;
}
void display_salary(){
System.out.println("The salary of the employee is "+salary);
}
}
class Programmer extends Employee{
int bonus;
Programmer(int salary,int bonus){
super(salary);
this.bonus=bonus;
}
void display_bonus(){
System.out.println("The bonus of the employee is "+bonus);
}
void total_income(){
System.out.println("The total income of the employee is "+(salary+bonus));
}
}
public class Prog1{
public static void main(String[] args){
Programmer p1=new Programmer(10000,3000);
p1.display_salary();
```

```
p1.display_bonus();

p1.total_income();

}

}
```

OUTPUT :

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
The salary of the employee is 10000
The bonus of the employee is 3000
The total income of the employee is 13000

C:\Users\Admin\Documents\Java>
```

5. The Question:

● Grandparent: Class Animal with a method eat().

● Parent: Class Dog (extends Animal) with a method bark().

● Child: Class BabyDog (extends Dog) with a method weep().Create an instance of BabyDog and invoke all three methods to demonstrate the chain of inheritance.

CODE :

```
class Animal{

void eat(){

System.out.println("Animal eats");

}

}

class Dog extends Animal{

void bark(){

System.out.println("Dog barks...");

}

}

class BabyDog extends Dog{

void weep(){
```

```
System.out.println("BabyDog weeps...");
}
}
public class Prog1{
public static void main(String[] args){
BabyDog b1=new BabyDog();
b1.eat();
b1.bark();
b1.weep();
}
}
```

OUTPUT:

```
C:\Users\Admin\Documents\Java>javac Prog1.java

C:\Users\Admin\Documents\Java>java Prog1
Animal eats
Dog barks...
BabyDog weeps...

C:\Users\Admin\Documents\Java>
```

6. Design a base class Shape with a method draw(). Create two subclasses, Circle and Rectangle, that override the draw() method to print specific messages (e.g., "Drawing a Circle"). In your main class, instantiate both and call their respective draw() methods.

CODE :

```
class Shape{
void draw(){
System.out.println("Drawing a shape..");
}
}
class Circle extends Shape{
```

```
void draw(){
System.out.println("Drawing a Circle...");
}
}
class Rectangle extends Shape{
void draw(){
System.out.println("Drawing a Rectangle...");
}
}
public class Prog1{
public static void main(String[] args){
Circle c1= new Circle();
c1.draw();
Rectangle r1= new Rectangle();
r1.draw();
}
}
```

OUTPUT :

```
C:\Users\admin\OneDrive\Documents>javac Prog1.java

C:\Users\admin\OneDrive\Documents>java Prog1
Drawing a Circle...
Drawing a Rectangle...

C:\Users\admin\OneDrive\Documents>
```

PRACTICAL 9

CODE :

Person.java

```
public class Person{
public String name;
private int age;
protected String jobTitle;
String address;

public Person(String name,int age,String jobTitle,String address){
this.name=name;
this.age=age;
this.jobTitle=jobTitle;
this.address = address;
}

public void displayPublicInfo(){
System.out.println("Name: "+name);
}

private void displayPrivateInfo(){
System.out.println("Age: "+age);
}

protected void displayProtectedInfo(){
System.out.println("JobTitle: "+jobTitle);
}
```

```
void displayDefaultInfo(){
System.out.println("Address: "+address);
}
}
```

EEmployee.java

```
public class EEmployee extends Person{
public EEmployee(String name,int age,String jobTitle,String address)
{
super(name,age,jobTitle,address);
}
public void displayEmployeeInfo(){
System.out.println("Employee name(from public field):"+name);
System.out.println("Employee jobTitle(from protected field):"+jobTitle);
System.out.println("Employee address(from default field):"+address);

displayPublicInfo();
displayProtectedInfo();
displayDefaultInfo();
}
}
```

TestAccess.java

```
public class TestAccess{
public static void main(String[] args){
Person person=new Person("John Doe",30,"Software Developer","1234 Elim Street");
System.out.println("Accessing public field from Person:"+person.name);
person.displayPublicInfo();
```

```
System.out.println("Accessing Protected field from Person:"+person.jobTitle);

person.displayProtectedInfo();


System.out.println("Accessing Default field from Person:"+person.address);

person.displayDefaultInfo();


EEmployee emp1 = new EEmployee("Jane Smith",28,"Project Manager","5678 Oak
Avenue");

emp1.displayEmployeeInfo();

}

}
```

OUTPUT :

```
C:\Users\Admin\Documents\Java>javac TestAccess.java

C:\Users\Admin\Documents\Java>java TestAccess
Accessing public field from Person:John Doe
Name: John Doe
Accessing Protected field from Person:Software Developer
JobTitle: Software Developer
Accessing Default field from Person:1234 Elim Street
Address: 1234 Elim Street
Employee name(from public field):Jane Smith
Employee jobTitle(from protected field):Project Manager
Employee address(from default field):5678 Oak Avenue
Name: Jane Smith
JobTitle: Project Manager
Address: 5678 Oak Avenue
```

Practical no. 10

Code 1:

Method Overloading :-

```
class Calculator{
public int add(int a,int b){
return a+b;
}
public double add(double a,double b){
return a+b;
}
public int add(int a,int b,int c){
return a+b+c;
}
}
class Prog10{
public static void main(String[] args){
Calculator c=new Calculator();
System.out.println("addition of two int num is "+c.add(10,20));
System.out.println("addition of two double num is "+c.add(10.5,20.5));
System.out.println("addition of three int num is "+c.add(10,20,30));
}
}
```

Output :

```
PS C:\Users\Admin\Documents\Java> javac Prog10.java
PS C:\Users\Admin\Documents\Java> java Prog10
addition of two int num is 30
addition of two double num is 31.0
addition of three int num is 60
PS C:\Users\Admin\Documents\Java>
```

code 2:

Method Overriding:-

```
class Animal{
public String sound(){
return "animal sound";
}
}
class Dog extends Animal{
@Override
public String sound(){
return "barks!!!";
}
}
class Cat extends Animal{
@Override
public String sound(){
return "meows!!!";
}
}
class Prog10{
public static void main(String[] args){
Dog d=new Dog();
Cat c=new Cat();
System.out.println(d.sound());
```

```
System.out.println(c.sound());
}
}
```

Output:

```
PS C:\Users\Admin\Documents\Java> javac Prog10.java
PS C:\Users\Admin\Documents\Java> java Prog10
barks!!!
meows!!!
PS C:\Users\Admin\Documents\Java>
```

Code 3:

Operator overloading:-

```
class Prog10{
public static void main(String[] args){
int n1=20;
int n2=30;
int total=n1+n2;
System.out.println("addition is "+total);
String name1="Sandhya";
String name2="Gupta";
String combine=name1+name2;
System.out.println("Concatenated word is "+name1+" "+name2);
System.out.println("Mixed types:"+"Score:"+100);


}
}
```

Output:

```
C:\Users\Admin\Documents\Java>javac Prog10.java

C:\Users\Admin\Documents\Java>java Prog10
addition is 50
Concatenated word is Sandhya Gupta
Mixed types:Score:100

C:\Users\Admin\Documents\Java>
```

Code 4:

Method Overloading – Area Calculation

Create a Java class named Shape.

Requirements:

1. Define a method area() that is overloaded to calculate:

○ Area of a square (one parameter: side)

○ Area of a rectangle (two parameters: length and breadth)

○ Area of a circle (one parameter: radius, use double)

2. Display the calculated area for each shape in the main() method.

```
class Shape{

public int area(int a){

return a*a;

}

public double area(double b,double c){

return b*c;

}

public double area(double r){
```

```
return 3.142*r*r;
}
}
class Prog10{
public static void main(String[] args){
Shape s=new Shape();
System.out.println("Area of square is: "+s.area(5));
System.out.println("Area of rectangle is: "+s.area(5.1,2.8));
System.out.println("Area of circle is: "+s.area(4.4));
}
}
```

Output:

```
C:\Users\Admin\Documents\Java>javac Prog10.java

C:\Users\Admin\Documents\Java>java Prog10
Area of square is: 25
Area of rectangle is: 14.279999999999998
Area of circle is: 60.82912000000001

C:\Users\Admin\Documents\Java>
```

Code 5:

Method Overriding – Bank Interest

Create a program to demonstrate method overriding.

Requirements:

1. Create a superclass Bank with a method getInterestRate() that returns 0.

2. Create subclasses:

- **SBI → returns 6.5**
- **ICICI → returns 7.0**

3. In the main() method:

- **Use a Bank reference to call getInterestRate() for each subclass.**
- **Print the interest rate.**

```
class Bank{
public double getInterestRate(){
return 0;
}
}
class SBI extends Bank{
@Override
public double getInterestRate(){
return 6.5;
}
}
class ICICI extends Bank{
@Override
public double getInterestRate(){
return 7.0;
}
}
```

```
class Prog10{
public static void main(String[] args){
SBI s=new SBI();
ICICI i=new ICICI();
System.out.println("Interest rate of sbi is: "+s.getInterestRate());
System.out.println("Interest rate of icici is: "+i.getInterestRate());
}
}
```

Output :

```
PS C:\Users\Admin\Documents\Java> javac Prog10.java
PS C:\Users\Admin\Documents\Java> java Prog10
Interest rate of sbi is: 6.5
Interest rate of icici is: 7.0
PS C:\Users\Admin\Documents\Java>
```

Practical 11 : Exception Handling

a. Write a java program that handles

IOException

ArithmeticException

ArrayIndexOutofBoundsException

ClassNotFoundException

NumberFormatException

NullPointerException

FileNotFoundException

Code :

```
import java.io.File;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.*;

public class Prac4{
public static void main(String[] args){
try{
int i = 10/0;
}
catch(ArithmeticException e){
  System.out.println(e);
     }
try{
```

```
int x[]=new int[7];
x[8]=15;
}
catch(ArrayIndexOutOfBoundsException e){
  System.out.println(e);
     }
try{
String name="Sandhya";
int b=Integer.parseInt(name);
   }
catch(NumberFormatException e){
  System.out.println(e);
     }
try{
String c=null;
System.out.println(c.length());
}
catch(NullPointerException e){
System.out.println(e);
}
try{
Class.forName("Myclass");
}
catch(ClassNotFoundException e){
System.out.println(e);
```

```
  }
try{
File f=new File("C:/Sandhya17.txt");
FileInputStream z=new FileInputStream(f);
  }
catch(FileNotFoundException e){
System.out.println(e);
  }
try{
  FileReader reader=new FileReader("Sandhya.txt");
  reader.close();
  reader.read();
}
catch(IOException e){
System.out.println(e);
}
finally{
System.out.println("Code ended");
    }
}
}
```

Output :

```
C:\Users\Admin\Documents\Java>javac Prac4.java

C:\Users\Admin\Documents\Java>java Prac4
java.lang.ArithmeticException: / by zero
java.lang.ArrayIndexOutOfBoundsException: Index 8 out of bounds for length 7
java.lang.NumberFormatException: For input string: "Sandhya"
java.lang.NullPointerException: Cannot invoke "String.length()" because "<local1>" is null
java.lang.ClassNotFoundException: Myclass
java.io.FileNotFoundException: C:\Sandhya17.txt (The system cannot find the file specified)
java.io.IOException: Stream closed
Code ended

C:\Users\Admin\Documents\Java>
```

b. Create a custom exception class NegativeNumberException. Write a program that reads an integer from the

user and throws this exception if the number is negative. Otherwise, print the square of the number.

Code :

```
import java.util.Scanner;


class NegativeNumberException extends Exception{
  public NegativeNumberException(String message){
    super(message);
  }
}


public class NegativeNumberTest{
public static void checkAndSquare(int number)
throws NegativeNumberException{
  if(number<0){
    throw new NegativeNumberException("Negative numbers are not allowed:"+number);
}
```

```
else{
System.out.println("The square of "+number+"is: "+(number*number));
}
}

public static void main(String[] args){
  Scanner scanner=new Scanner(System.in);
 try{
System.out.print("Enter an integer:");
int number=scanner.nextInt();

checkAndSquare(number);
}

catch(NegativeNumberException e){
  System.out.println("Exception:"+e.getMessage());
}
catch(Exception e){
  System.out.println("Invalid input.Please enter an integer.");
}
finally{
scanner.close();
System.out.println("Program execution completed.");
}
}
```

}

Output :

```
C:\Users\Admin\Documents\Java>java NegativeNumberTest
Enter an integer:4
The square of 4is: 16
Program execution completed.

C:\Users\Admin\Documents\Java>java NegativeNumberTest
Enter an integer:-8
Exception:Negative numbers are not allowed:-8
Program execution completed.

C:\Users\Admin\Documents\Java>
```

c.

Write a program that performs the following operations:

Read an integer from the user.

Perform division by another number.

Handle potential exceptions (NumberFormatException, ArithmeticException) using nested try-catch blocks.

Code :

```
import java.util.Scanner;

public class Question3 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int numerator = 0;

        try {
            System.out.print("Enter an integer: ");
            String input = scanner.nextLine();
            numerator = Integer.parseInt(input);
```

```
        try {
            System.out.print("Enter a divisor: ");
            int divisor = scanner.nextInt();
            int result = numerator / divisor;
            System.out.println("Result: " + result);
        } catch (ArithmeticException ae) {
            System.out.println("Error: Division by zero is not allowed.");
        }

    } catch (NumberFormatException nfe) {
        System.out.println("Error: Invalid input. Please enter a valid integer.");
    }
  }
}
```

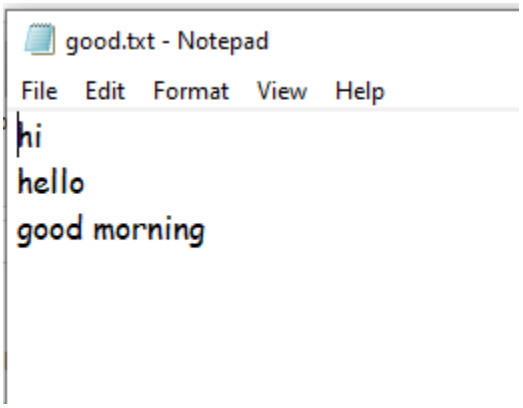
Output:

```
C:\Users\Admin\Documents\Java>javac NegativeNumberTest.java

C:\Users\Admin\Documents\Java>java NegativeNumberTest
Enter an integer: 5
Enter a divisor: 0
Error: Division by zero is not allowed.

C:\Users\Admin\Documents\Java>
```

PRACTICAL 12 : FILE HANDLING

File handling

First make a file good.txt and save all the text files and code in one folder only.



Code:

```
import java.io.FileInputStream;

public class Files {

        public static void main(String args[]) {

                try{

                        FileInputStream fin=new FileInputStream("good.txt");

                        int i=fin.read();

                        System.out.print((char)i);

                        fin.close();

                }


                catch(Exception e){System.out.println(e);

}

                }
```

```
}
```

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\student>cd C:\Users\student\Desktop\java moni\file handling

C:\Users\student\Desktop\java moni\file handling>javac Files.java

C:\Users\student\Desktop\java moni\file handling>java Files
h
C:\Users\student\Desktop\java moni\file handling>
```

,

2**. prints all characters**

```
import java.io.FileInputStream;
public class Allchar {
        public static void main(String args[]) {
                try{
                        FileInputStream fin=new FileInputStream("good.txt");
                        int i=0;
                        while((i=fin.read())!=-1){
                        System.out.print((char)i);

                }
                fin.close();
                } catch(Exception e){System.out.println(e);}
                }
                }
```

```
C:\Users\student\Desktop\java moni\file handling>javac Allchar.java

C:\Users\student\Desktop\java moni\file handling>java Allchar
hi
hello
good morning
```

3. FileOutputStream

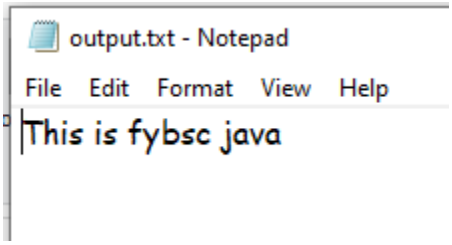
```
import java.io.FileOutputStream;
import java.io.OutputStream;

public class Outputstream {
	public static void main(String args[]) {
		String data ="This is fybsc java";

		try{
			OutputStream out = new FileOutputStream("output.txt");
			byte[] dataBytes = data.getBytes();
			out.write(dataBytes);
			System.out.print("data is written to the file.");
			out.close();
		}

		catch(Exception e){
```

```
                    e.getStackTrace();
            }
        }
}
```

```
C:\Users\student\Desktop\java moni\file handling>javac Outputstream.java

C:\Users\student\Desktop\java moni\file handling>java Outputstream
data is written to the file.
C:\Users\student\Desktop\java moni\file handling>
```

U will see output.txt file in your folder after running the code.

Name	Date modified	Type	Size
Allchar.class	2/3/2026 1:34 PM	CLASS File	1 KB
Allchar.java	2/3/2026 1:34 PM	Java Source File	1 KB
Files.class	2/3/2026 1:27 PM	CLASS File	1 KB
Files.java	2/3/2026 1:26 PM	Java Source File	1 KB
good.txt	2/3/2026 1:23 PM	Text Document	1 KB
output.txt	2/3/2026 1:45 PM	Text Document	1 KB
Outputstream.class	2/3/2026 1:44 PM	CLASS File	1 KB
Outputstream.java	2/3/2026 1:44 PM	Java Source File	1 KB



4. BufferedWriter

```
import java.io.FileWriter;
```

```
import java.io.BufferedWriter;

public class Bufferw {
	public static void main(String args[]) {
		String data = "This is adishree";

		try {
			FileWriter file = new FileWriter("name.txt");

			BufferedWriter output = new BufferedWriter(file);

			output.write(data);
			output.close();
		}

		catch(Exception e) {
			e.getStackTrace();
		}
	}
}
```

U will see name.txt file in your folder .

Name	Date modified	Type	Size
Allchar.class	2/3/2026 1:34 PM	CLASS File	1 KB
Allchar.java	2/3/2026 1:34 PM	Java Source File	1 KB
Bufferw.class	2/3/2026 2:09 PM	CLASS File	1 KB
Bufferw.java	2/3/2026 2:08 PM	Java Source File	1 KB
Files.class	2/3/2026 1:27 PM	CLASS File	1 KB
Files.java	2/3/2026 1:26 PM	Java Source File	1 KB
good.txt	2/3/2026 1:23 PM	Text Document	1 KB
name.txt	2/3/2026 2:09 PM	Text Document	1 KB
output.txt	2/3/2026 1:45 PM	Text Document	1 KB
Outputstream.class	2/3/2026 1:44 PM	CLASS File	1 KB
Outputstream.java	2/3/2026 1:44 PM	Java Source File	1 KB

name.txt - Notepad

File Edit Format View Help

This is adishree

5. Buffered Reader

```
import java.io.FileReader;
import java.io.BufferedReader;

class Bufferr {
	public static void main(String args[]) {

	char[] array = new char[100];
	try {
		FileReader file = new FileReader("good.txt");
		BufferedReader input=new BufferedReader(file);
```

```
            input.read(array);
            System.out.println("data in the file :");
            System.out.println(array);

            input.close();
            }

            catch(Exception e) {
                e.getStackTrace();
            }
        }
}
```

```
C:\Users\student\Desktop\java moni\file handling>javac Bufferr.java

C:\Users\student\Desktop\java moni\file handling>java Bufferr
data in the file :
hi
hello
good morning
```

1. **Create a File.**

```
import java.io.File;
import java.io.IOException;
public class CreateFile
{
```

```
    public static void main(String[] args)
    {
        try {
            File Obj = new File("myfile.txt");

            if(Obj.createNewFile()) {
                System.out.println("File created :"+ Obj.getName()) ;
            }
        else {
            System.out.println("File already exists.");
        }
    }
        catch(IOException e) {
            System.out.println("an error has occurred.");
            e.printStackTrace();
        }
    }
}
```

```
C:\Users\student\Desktop\java moni\file handling>javac CreateFile.java

C:\Users\student\Desktop\java moni\file handling>java CreateFile
File created :myfile.txt
```

2. Write to a File

```
import java.io.FileWriter;
import java.io.IOException;
public class WriteFile
{

	public static void main(String[] args)
	{
		try {
			FileWriter Writer = new FileWriter("myfile.txt");
			Writer.write("files in java are seriously good!!");
			Writer.close();

			System.out.println("successfully written.");
		}

		catch(IOException e) {
			System.out.println("an error has occurred.");
			e.printStackTrace();
		}
	}
}
```

```
C:\Users\student\Desktop\java moni\file handling>javac WriteFile.java

C:\Users\student\Desktop\java moni\file handling>java WriteFile
successfully written.
```

myfile.txt - Notepad

File Edit Format View Help

files in java are seriously good!!

3. Read from a File

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;
public class ReadFile
{

	public static void main(String[] args)
	{
		try {
			File Obj = new File("myfile.txt");
			Scanner Reader = new Scanner(Obj);
			while(Reader.hasNextLine()){
				String data=Reader.nextLine();
				System.out.println(data);
			}
			Reader.close();
```

```
            }

            catch(FileNotFoundException e) {
                System.out.println("an error has occurred.");
                e.printStackTrace();
            }
        }
}
```

```
C:\Users\student\Desktop\java moni\file handling>javac ReadFile.java

C:\Users\student\Desktop\java moni\file handling>java ReadFile
files in java are seriously good!!
```

4. Delete a File

```
import java.io.File;
public class DeleteFile
{

        public static void main(String[] args)
        {
                File Obj = new File("myfile.txt");
                if(Obj.delete()) {
                        System.out.println("the deleted file is:"
+Obj.getName());
                }
```

```
                    else {
                        System.out.println("failed in deleting the file");


                    }
}
}
```

```
C:\Users\student\Desktop\java moni\file handling>javac DeleteFile.java

C:\Users\student\Desktop\java moni\file handling>java DeleteFile
the deleted file is:myfile.txt
```

PRACTICAL 13:SERVER-CLIENT CONNECTION

CODE :

Server.java

```
import java.net.*;

import java.io.*;

public class Server{

public static void main(String args[]){

try{

ServerSocket serverSocket= new ServerSocket(1234);

System.out.println("Server is waiting for client connection.....");

Socket socket=serverSocket.accept();

System.out.println("client Connected!");

BufferedReader input = new BufferedReader(new
InputStreamReader(socket.getInputStream()));

PrintWriter output=new PrintWriter(socket.getOutputStream(),true);


String message= input.readLine();

System.out.println("Client Says: "+message);

output.println("hello from server!");


input.close();

output.close();

socket.close();
```

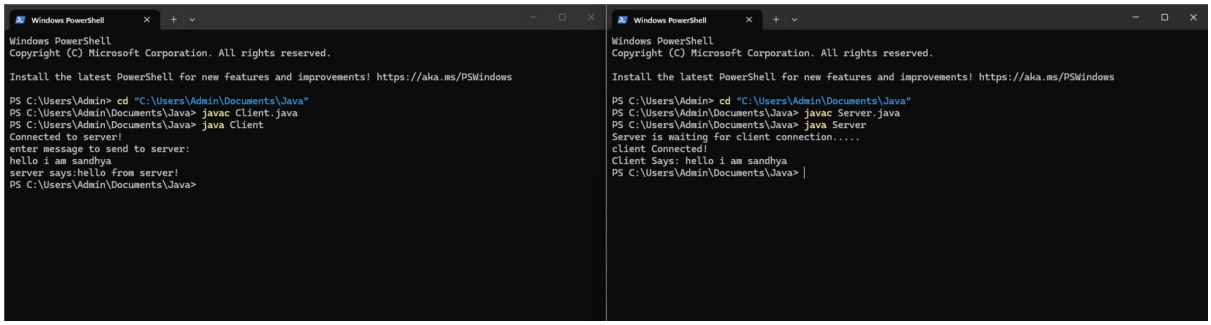
```
serverSocket.close();
}
catch(IOException e){
e.printStackTrace();
}
}
}
```

Client.java

```
import java.io.*;
import java.net.*;
public class Client{
public static void main(String[] args){
try{
Socket socket= new Socket("Localhost",1234); //1234 is port number
System.out.println("Connected to server!");
BufferedReader userInput= new BufferedReader(new
InputStreamReader(System.in));
PrintWriter output= new PrintWriter(socket.getOutputStream(),true);
BufferedReader serverInput= new BufferedReader(new
InputStreamReader(socket.getInputStream()));
System.out.println("enter message to send to server:");
String message= userInput.readLine();
output.println(message);
```

```
//read response from server
String response=serverInput.readLine();
System.out.println("server says:"+response);


userInput.close();
output.close();
serverInput.close();
}
catch(IOException e){
e.printStackTrace();
}
}
}
```

Output:



getInputStream() is used to take the input from the file

getOutputStream() is used to give the output to a file

2) Connecting Server-client on two different Computer

Code :

RemoteServer.java

```
import java.io.*;
import java.net.*;
public class RemoteServer {
    public static void main(String[] args) {
        try (ServerSocket serverSocket = new
ServerSocket(5000)) {
            System.out.println("Server started. Waiting for a
client...");

            Socket socket = serverSocket.accept();
            System.out.println("Client connected from: " +
socket.getInetAddress());

            PrintWriter out = new
PrintWriter(socket.getOutputStream(), true);
            out.println("Connection Successful! You are talking
to PC A.");

            socket.close();
        } catch (IOException e) {
```

```
            e.printStackTrace();
        }
    }
}
```

RemoteClient.java

```
import java.io.*;
import java.net.*;

public class RemoteClient {
    public static void main(String[] args) {
        // REPLACE THIS IP with the IP of your Server PC
        String serverIP = "192.168.1.15";
        int port = 5000;

        try (Socket socket = new Socket(serverIP, port)) {
            BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));

            // Read the welcome message from the server
            String response = in.readLine();
            System.out.println("Server says: " + response);
```

```
        } catch (IOException e) {
            System.out.println("Could not connect. Check IP and Firewall!");
            e.printStackTrace();
        }
    }
}
```

Output:

Server:

```
Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Admin> cd "C:\Users\Admin\Documents\Java"
PS C:\Users\Admin\Documents\Java> javac RemoteServer.java
PS C:\Users\Admin\Documents\Java> java RemoteServer
Server started. Waiting for a client...
Client connected from: /192.168.2.244
PS C:\Users\Admin\Documents\Java>
```

Client:

```
C:\Users\Admin\Documents\java>javac RemoteClient.java

C:\Users\Admin\Documents\java>java RemoteClient
Server says: Connection Successful! You are talking to PC A.

C:\Users\Admin\Documents\java>
```

3) Chatting

Code :

RemoteServer.java

```
import java.io.*;

import java.net.*;

public class RemoteServer {

    public static void main(String[] args) {

        try (ServerSocket serverSocket = new
ServerSocket(5000)) {

            System.out.println("Server started. Waiting for a
client...");


            Socket socket=serverSocket.accept();

System.out.println("client Connected!");

BufferedReader input = new BufferedReader(new
InputStreamReader(socket.getInputStream()));

PrintWriter output=new
PrintWriter(socket.getOutputStream(),true);


String message= input.readLine();

System.out.println("Client Says: "+message);

output.println("hello from server!");
```

```
input.close();
output.close();
socket.close();
serverSocket.close();
}
      catch (IOException e){
            e.printStackTrace();
        }
    }
}
```

RemoteClient.java

```
import java.io.*;
import java.net.*;

public class RemoteClient {
    public static void main(String[] args) {
        // REPLACE THIS IP with the IP of your Server PC
        String serverIP = "192.168.2.248";
        int port = 5000;

        try (Socket socket = new Socket(serverIP, port)) {
            BufferedReader userInput= new
```

```
BufferedReader(new InputStreamReader(System.in));
PrintWriter output= new
PrintWriter(socket.getOutputStream(),true);
BufferedReader serverInput= new BufferedReader(new
InputStreamReader(socket.getInputStream()));
System.out.println("enter message to send to server:");
String message= userInput.readLine();
output.println(message);

//read response from server
String response=serverInput.readLine();
System.out.println("server says:"+response);

userInput.close();
output.close();
serverInput.close();

      } catch (IOException e) {
          System.out.println("Could not connect. Check IP and
Firewall!");
          e.printStackTrace();
       }
   }
}
```

Output :

```
C:\Users\Admin\Documents\Java>javac RemoteServer.java

C:\Users\Admin\Documents\Java>java RemoteServer
Server started. Waiting for a client...
client Connected!
Client Says: hiii

C:\Users\Admin\Documents\Java>
```

```
C:\Users\Admin\Documents\java>javac RemoteClient.java

C:\Users\Admin\Documents\java>java RemoteClient
enter message to send to server:
hiii
server says:hello from server!
```

PRACTICAL 14

Programs:

1) You can create a thread in two primary ways:

- By Extending the Thread class: You can create a new thread by subclassing the Thread

class and overriding its run() method.

Code:

```
class MyThread extends Thread {
@Override
public void run() {
System.out.println("Thread is running");
}
}

public class ThreadExample {
public static void main(String[] args) {

MyThread thread = new MyThread();
thread.start(); // starts the new thread
}
}
```

Output:

```
C:\Users\Admin\Downloads\New folder>javac ThreadExample.java

C:\Users\Admin\Downloads\New folder>java ThreadExample
Thread is running
```

2) By Implementing the Runnable Interface: You can also create a thread by implementing the Runnable interface and passing it to a Thread object.

Code:

```
class MyRunnable implements Runnable {

@Override

public void run() {

System.out.println("Thread is running");

}

}


public class ThreadExample2 {

public static void main(String[] args) {

MyRunnable runnable = new MyRunnable();

Thread thread = new Thread(runnable);

thread.start(); // starts the new thread

}


}
```

Output:

```
C:\Users\Admin\Downloads\New folder>javac ThreadExample2.java

C:\Users\Admin\Downloads\New folder>java ThreadExample2
Thread is running

C:\Users\Admin\Downloads\New folder>
```

3) Code for Multithreading

Here's an example that demonstrates the use of threads and synchronization in a simple bank

account transfer scenario:

Code:

```
class Account {
    private int balance = 1000;


    // Synchronized method to ensure thread safety
    public synchronized void transfer(Account target, int amount) {
        if (this.balance >= amount) {
            this.balance -= amount;
            target.deposit(amount);
```

```
            System.out.println("Transferred " + amount + " to target
account");
        } else {
            System.out.println("Insufficient funds for transfer");
        }
    }

    // Synchronized method to ensure thread safety
    public synchronized void deposit(int amount) {
        this.balance += amount;
    }

    public int getBalance() {
        return balance;
    }
}

public class BankExample {
    public static void main(String[] args) {
        Account account1 = new Account();
        Account account2 = new Account();
```

```
        Thread t1 = new Thread(() -> account1.transfer(account2,
500));
        Thread t2 = new Thread(() -> account2.transfer(account1,
300));


        t1.start();
        t2.start();


        try {
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }


        System.out.println("Account1 balance: " +
account1.getBalance());
        System.out.println("Account2 balance: " +
account2.getBalance());
    }
}
```

Output:

```
C:\Users\Admin\Downloads\New folder>javac BankExample.java

C:\Users\Admin\Downloads\New folder>java BankExample
Transferred 500 to target account
Transferred 300 to target account
Account1 balance: 800
Account2 balance: 1200

C:\Users\Admin\Downloads\New folder>
```

Problems:

1) Create a Simple Thread

- Write a Java program that creates a thread using the Thread class and prints “Hello,World!”; when the thread runs.

Code:

```
class HelloThread extends Thread {
    @Override
    public void run() {
        System.out.println("Hello, World!");
    }
}


public class Prob1 {
    public static void main(String[] args) {
        HelloThread thread = new HelloThread();
        thread.start();
    }
}
```

Output:

```
C:\Users\Admin\Downloads\New folder>javac Prob1.java

C:\Users\Admin\Downloads\New folder>java Prob1
Hello, World!
```

2) Use of Runnable Interface

- Implement a class that implements the Runnable interface. The thread should print

“Thread is running”; five times with a small delay (e.g., 1 second between prints).

Code:

```
class MyRunnable implements Runnable {
    @Override
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Thread is running");
            try {
                Thread.sleep(1000); // 1 second delay
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

public class RunnableExample {
    public static void main(String[] args) {
        Thread thread = new Thread(new MyRunnable());
        thread.start();
```

```
    }
}
```

Output:

```
C:\Users\Admin\Downloads\New folder>javac Prob2.java

C:\Users\Admin\Downloads\New folder>java Prob2
Thread is running
Thread is running
Thread is running
Thread is running
Thread is running
```

3)Thread Priority

- Write a Java program that creates two threads with different priorities and prints the order in which they execute (e.g., Thread.MAX_PRIORITY vs. Thread.MIN_PRIORITY).

Code:

```
class PriorityThread extends Thread {
    public PriorityThread(String name) {
        super(name);
    }

    @Override
    public void run() {
        System.out.println(
            "Running: " + getName() +
            " | Priority: " + getPriority()
        );
```

```
    }
}

public class ThreadPriorityExample {
    public static void main(String[] args) {
        PriorityThread t1 = new PriorityThread("High Priority Thread");
        PriorityThread t2 = new PriorityThread("Low Priority Thread");

        t1.setPriority(Thread.MAX_PRIORITY);
        t2.setPriority(Thread.MIN_PRIORITY);

        t1.start();
        t2.start();
    }
}
```

Output:

```
C:\Users\Admin\Downloads\New folder>javac Prob3.java

C:\Users\Admin\Downloads\New folder>java Prob3
Running: Low Priority Thread | Priority: 1
Running: High Priority Thread | Priority: 10

C:\Users\Admin\Downloads\New folder>
```

4) Using Thread.sleep()

- Create two threads where one thread prints a message every 2 seconds and the other

thread prints a message every 3 seconds. Use Thread.sleep() to delay the execution.

Code:

```
class TwoSecondThread extends Thread {
    @Override
    public void run() {
        while (true) {
            System.out.println("Message every 2 seconds");
            try {
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                break;
            }
        }
    }
}

class ThreeSecondThread extends Thread {
    @Override
    public void run() {
        while (true) {
```

```
            System.out.println("Message every 3 seconds");
            try {
                Thread.sleep(3000);
            } catch (InterruptedException e) {
                break;
            }
        }
    }
}

public class SleepExample {
    public static void main(String[] args) {
        new TwoSecondThread().start();
        new ThreeSecondThread().start();
    }
}
```

Output:

```
C:\Users\Admin\Downloads\New folder>javac Prob4.java

C:\Users\Admin\Downloads\New folder>java Prob4
Message every 2 seconds
Message every 3 seconds
Message every 2 seconds
Message every 3 seconds
Message every 2 seconds
Message every 3 seconds
Message every 2 seconds
Message every 2 seconds
Message every 3 seconds
Message every 2 seconds
Message every 3 seconds
Message every 2 seconds
Message every 2 seconds
Message every 3 seconds
Message every 2 seconds
Message every 3 seconds
Message every 2 seconds
```

5) Implement Counter Without Synchronization

- Write a program with a shared counter variable and two threads that increment it.

Observe the race condition problem and incorrect results.

Code:

```
class Counter {
    int count = 0;


    public void increment() {
        count++;
    }
}
```

```
public class RaceConditionExample {
    public static void main(String[] args) throws
InterruptedException {
        Counter counter = new Counter();

        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                counter.increment();
            }
        });

        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                counter.increment();
            }
        });

        t1.start();
        t2.start();

        t1.join();
        t2.join();
```

```
        System.out.println("Final count (incorrect): " +
counter.count);
    }
}
```

Output:

```
C:\Users\Admin\Downloads\New folder>java Prob5
Final count (incorrect): 1993

C:\Users\Admin\Downloads\New folder>
```

6)

Code:

```
class SynchronizedCounter {
    int count = 0;

    public synchronized void increment() {
        count++;
    }
}

public class SynchronizedCounterExample {
    public static void main(String[] args) throws
InterruptedException {
        SynchronizedCounter counter = new
SynchronizedCounter();
```

```
        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                counter.increment();
            }
        });

        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                counter.increment();
            }
        });

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("Final count (correct): " +
counter.count);
    }
```

}

Output:

```
C:\Users\Admin\Downloads\New folder>javac Prob6.java

C:\Users\Admin\Downloads\New folder>java Prob6
Final count (correct): 2000
```

Practical 15: Abstract classes and Interfaces in Java

1. Vehicle, Car and Bike:
   Code:
```
public class Car_Bike {
    abstract class Vehicle {
        abstract void start();

        void stop() {
            System.out.println("The vehicle has stopped!");
        }
    }

    class Car extends Vehicle {
        @Override
        void start() {
            System.out.println("The Bike has started!");
        }
    }

    class Bike extends Vehicle {
        @Override
        void start() {
            System.out.println("The Bike has started!");
        }
    }

    public static void main(String[] args) {
        Car_Bike parent = new Car_Bike();

        Car myCar = parent.new Car();
        Bike myBike = parent.new Bike();

        System.out.println("\n=====Car=====\n");
        myCar.start();
        myCar.stop();

        System.out.println("\n=====Bike=====\n");
        myBike.start();
        myBike.stop();
    }
}
```

   Output:

```
=====Car=====

The Bike has started!
The vehicle has stopped!

=====Bike=====

The Bike has started!
The vehicle has stopped!
PS C:\Users\Admin\Documents\FY4220_Second_Semester\SEC_OOP_with_java_FY4220\Practical 15>
```

2. Shape, Circle and Rectangle:
   Code:

```
import java.util.Scanner;

abstract class Shape {
        abstract double area();
    }


class Circle extends Shape {
    double radius;

    Circle(double radius) {
        this.radius = radius;
    }

    @Override
    double area() {
        return Math.PI * radius * radius;
    }
}


class Rectangle extends Shape {
    double length, breadth;

    Rectangle(double length, double breadth) {
        this.length = length;
        this.breadth = breadth;
    }

    @Override
    double area() {
        return length * breadth;
    }
}
```

```
public class Shape_Area {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("\nEnter radius of circle: ");
        double radius = sc.nextDouble();
        Shape circle = new Circle(radius);
        System.out.printf("\nCircle area: %.2f%n\n", circle.area());

        System.out.print("\nEnter length of rectangle: ");
        double length = sc.nextDouble();
        System.out.print("\nEnter breadth of rectangle: ");
        double breadth = sc.nextDouble();
        Shape rectangle = new Rectangle(length, breadth);
        System.out.printf("\nRectangle area: %.2f%n\n", rectangle.area());


    }
}
```

Output:

```
Enter radius of circle: 12.56

Circle area: 495.60


Enter length of rectangle: 14.5

Enter breadth of rectangle: 10

Rectangle area: 145.00

PS C:\Users\Admin\Documents\FY4220_Second_Semester\SEC_OOP_with_java_FY422
0\Practical 15>
```

3. Player:
   Code:

```
interface Playable {
    void play();
    void pause();
}

class MusicPlayer implements Playable {
```

```
    @Override
    public void play() {
        System.out.println("Playing music");
    }

    @Override
    public void pause() {
        System.out.println("Music paused");
    }
}

class VideoPlayer implements Playable {
    @Override
    public void play() {
        System.out.println("Playing video");
    }

    @Override
    public void pause() {
        System.out.println("Video paused");
    }
}

public class MainPlayable {
    public static void main(String[] args) {
        Playable music = new MusicPlayer();
        Playable video = new VideoPlayer();

        music.play();
        music.pause();

        video.play();
        video.pause();
    }
}
```

Output:

```
Playing music
Music paused
Playing video
Video paused
PS C:\Users\Admin\Documents\FY4220_Second_Semester\SEC_OOP_with_java_FY422
0\Practical 15>
```

4. Printer:

Code:

```
interface Printable {
    void print();
}

class Document implements Printable {
    @Override
    public void print() {
        System.out.println("Printing document...");
    }
}

class Photo implements Printable {
    @Override
    public void print() {
        System.out.println("Printing photo...");
    }
}

public class MainPrintable {
    public static void main(String[] args) {
        Printable doc = new Document();
        Printable photo = new Photo();

        doc.print();
        photo.print();
    }
}
```

Output:

```
Printing document...
Printing photo...
PS C:\Users\Admin\Documents\FY4220_Second_Semester\SEC_OOP_with_java_FY422
0\Practical 15>
```

5. Example1:

Code:

```
abstract class Animal {
 abstract void makeSound();
}

class Dog extends Animal {
 void makeSound() {
 System.out.println("Bark");
 }
```

```
}

public class Example1 {
 public static void main(String[] args) {
 Animal dog = new Dog();
 dog.makeSound();
 }
}
```

Output:

```
PS C:\Users\Admin\Documents\FY4220_Second_Semester\SEC_OOP_with_java_FY422
0\Practical 15> javac Example1.java
>> java Example1
Bark
PS C:\Users\Admin\Documents\FY4220_Second_Semester\SEC_OOP_with_java_FY422
0\Practical 15>
```

6. Example 2:

Code:

```
abstract class Animal {
 abstract void sound();
 void breathe() {
 System.out.println("Breathing...");
 }
}

class Dog extends Animal {
 @Override
 void sound() {
 System.out.println("Bark");
 }
}

public class Example2 {
 public static void main(String[] args) {
 Dog dog = new Dog();
 dog.breathe();
 dog.sound();
 }
}
```

Output:

```
PS C:\Users\Admin\Documents\FY4220_Second_Semester\SEC_OOP_with_java_FY422
0\Practical 15> javac Example2.java
>> java Example2
Breathing...
Bark
PS C:\Users\Admin\Documents\FY4220_Second_Semester\SEC_OOP_with_java_FY422
0\Practical 15>
```

7. Example 3:

Code:

```
interface Animal {
 void sound();
}

interface Playable {
 void play();
}

class Dog implements Animal, Playable {
 @Override
 public void sound() {
 System.out.println("Bark");
 }

 @Override
 public void play() {
 System.out.println("Playing fetch!");
 }
}

public class Example3 {
 public static void main(String[] args) {
 Dog dog = new Dog();
 dog.sound();
 dog.play();
 }
}
```

Output:

```
0\Practical 15> javac Example3.java
>> java Example3
Bark
Playing fetch!
PS C:\Users\Admin\Documents\FY4220_Second_Semester\SEC_OOP_with_java_FY422
0\Practical 15>
```

8. Example 4:

Code:

```
abstract class Vehicle {
 abstract void startEngine();
 void fuelEfficiency() {
 System.out.println("Fuel efficiency is 25 miles per gallon");
 }
}

interface Driveable {
 void accelerate();
 void brake();
}

class Car extends Vehicle implements Driveable {
 void startEngine() {
 System.out.println("Engine started");
 }

 public void accelerate() {
 System.out.println("Accelerating");
 }

 public void brake() {
 System.out.println("Braking");
 }
}

public class Example4 {
 public static void main(String[] args) {
 Car car = new Car();
 car.startEngine();
 car.fuelEfficiency();
 car.accelerate();
 car.brake();
 }
}
```

Output:

```
Engine started
Fuel efficiency is 25 miles per gallon
Accelerating
Braking
PS C:\Users\Admin\Documents\FY4220_Second_Semester\SEC_OOP_with_java_FY4220\Practical 15>
```